

Projet de Simulation Medium Access Control

GE Shuning - GUO Xuxin

1. Description du système et modélisation

Ce projet a pour objectif de simuler un protocole MAC utilisant l'algorithme Exponential Backoff afin d'étudier l'impact des collisions et des paramètres du système sur les performances réseau.

Le système étudié est composé de N stations qui partagent un unique canal de communication. Lorsque deux stations émettent simultanément, leurs paquets entrent en collision et sont reprogrammés. Pour gérer ces conflits de manière décentralisée, chaque station applique l'algorithme Exponential Backoff.

Paramètres du modèle

N : nombre de stations (référence : 5)
K : capacité de la file d'attente par station (référence : 10)
lambda : taux d'arrivée des paquets par station (référence : 1.0)
tau : paramètre du backoff (référence : 1.0)
T : durée de la simulation (référence : 1000)

Règles de fonctionnement

Les inter-arrivées suivent une loi exponentielle de paramètre lambda. La durée d'émission d'un paquet est fixée à 1 unité de temps.

Si deux émissions se recouvrent temporellement, il y a collision : les paquets sont perdus et reprogrammés après un délai de backoff.

Après une collision en état i, délai de retransmission distribué selon $\text{Exp}(1 / (2^i * \tau))$; la station passe à l'état i+1.

Après une émission réussie, la station retourne en état 1.

Un paquet arrivant sur une file pleine (K paquets) est perdu définitivement.

2. Éléments

2.1 Événements

ARRIVAL : arrivée d'un paquet sur la station. Planifie le prochain ARRIVAL et, si la station est libre, planifie immédiatement START_TX.

START_TX : début d'émission. Vérifie si d'autres stations sont en train d'émettre (détection de collision). Planifie END_TX dans 1 unité de temps.

END_TX : fin d'émission. Si collision : planifie retransmission après délai exponentiel (backoff). Si succès : retire le paquet de la file, remet l'état à 1, planifie la prochaine émission si la file n'est pas vide.

2.2 Variables d'état

queues[i] : nombre de paquets en file pour la station i.

backoff_state[i] : état courant du backoff de la station i (init. à 1).

transmitting[i] : booléen, True si la station i émet actuellement.

collided[i] : booléen, True si l'émission en cours est en collision.

scheduled_start[i] : booléen, évite de planifier deux START_TX consécutifs pour la même station.

2.3 Traitement des événements et métriques

La file d'événements est un tas (heapq). A chaque événement traité, les métriques sont mises à jour

: débit instantané $n(t)/t$, nombre moyen de clients dans le système, taux de paquets perdus. La détection de collision se fait au START_TX : si au moins une autre station a transmitting=True, toutes les stations impliquées sont marquées collided=True.

3. Hypothèses ajoutées par rapport a l'énoncé

Distribution du délai de retransmission : l'énoncé spécifie une loi exponentielle de paramètre $1/(2^i * \tau)$. Nous utilisons directement `numpy.random.exponential(2i * tau)`, ce qui est équivalent.

Détection des collisions au START_TX : deux paquets entrent en collision si leurs périodes d'émission se chevauchent. Comme la durée est fixé à 1 et que les transmissions ne sont pas interrompues, il suffit de vérifier `transmitting[other]=True` au moment du START_TX.

Le paquet qui subit une collision n'est pas perdu : il reste dans la file et est retransmis après le délai de backoff. Seuls les paquets qui trouvent la file pleine à leur arrivée sont définitivement perdus.

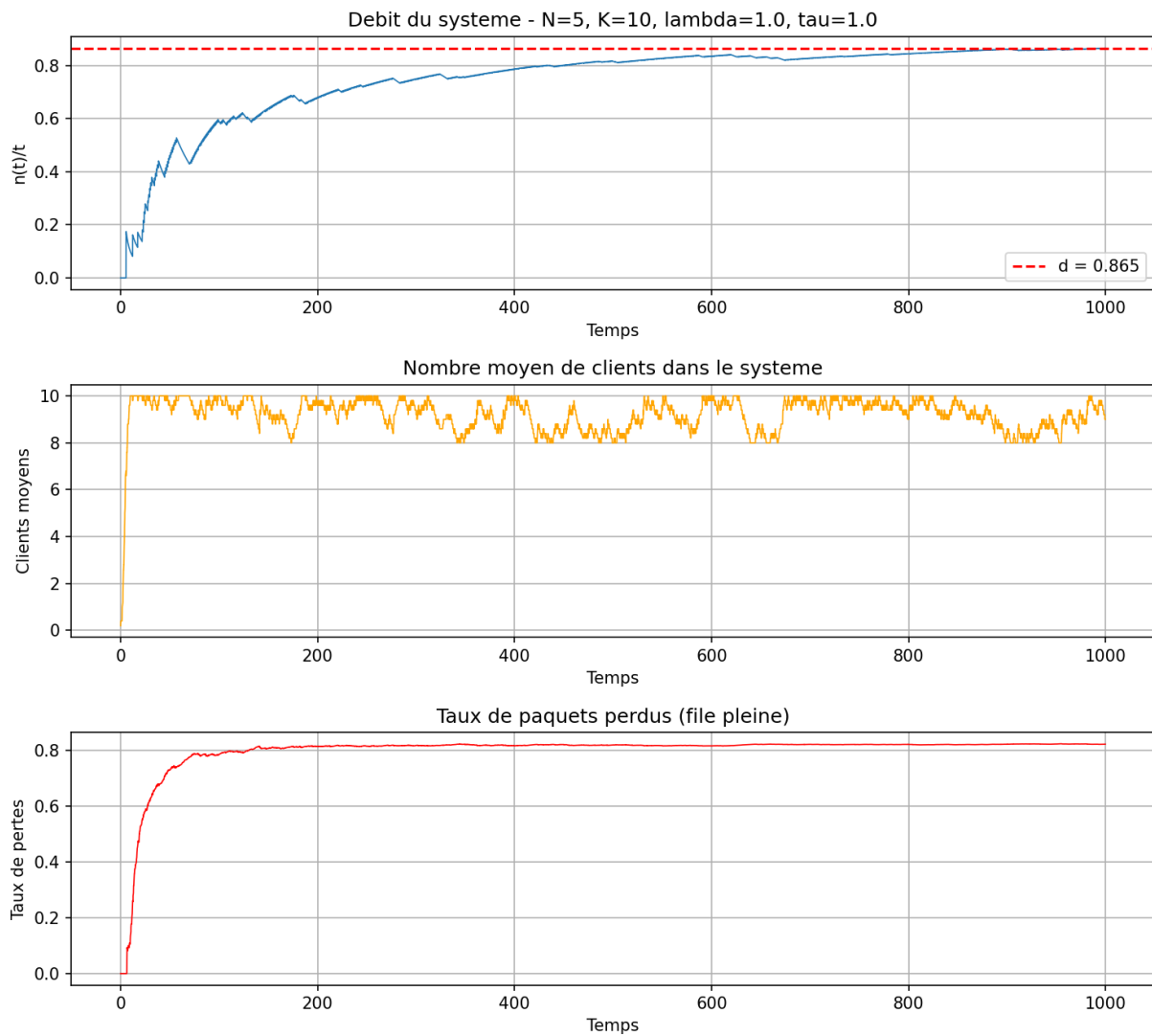
File d'attente FIFO de capacité K : un paquet arrivant sur une file pleine est immédiatement rejeté et comptabilisé dans le taux de perte.

L'état du backoff n'est réinitialisé à 1 qu'après une émission réussie, pas après un simple changement de paquet en attente.

4. Courbes demandées

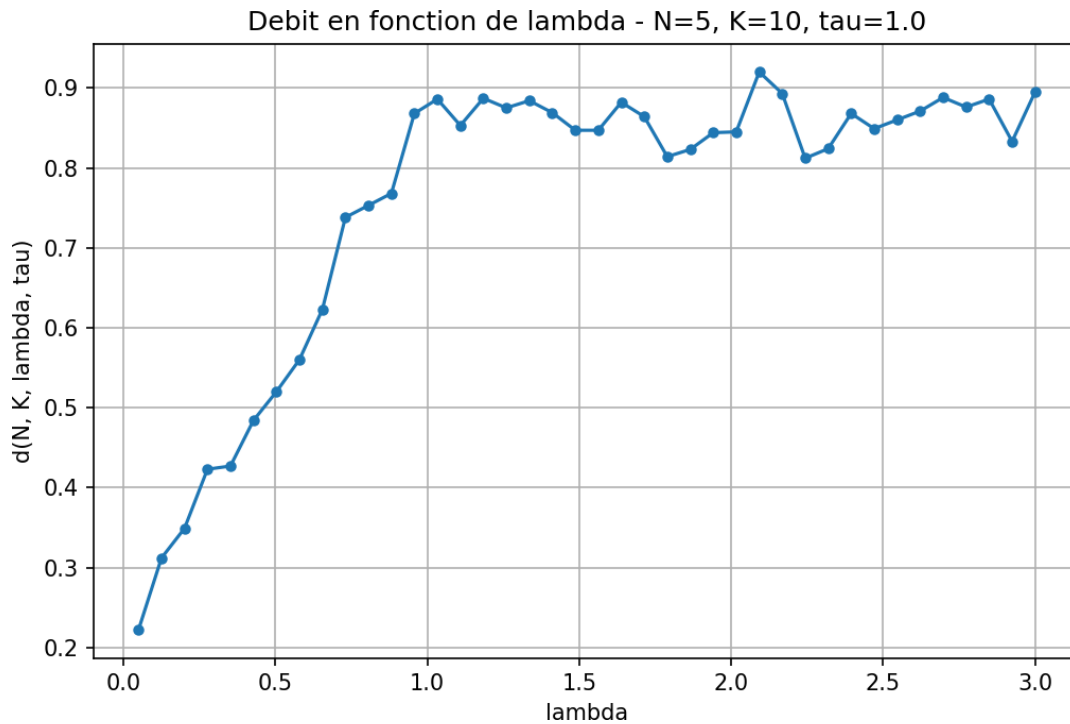
4.1 Évolution temporelle (N=5, K=10, lambda=1.0, tau=1.0)

Les trois courbes ci-dessous montrent l'évolution du débit $n(t)/t$, du nombre moyen de clients et du taux de paquets perdus en fonction du temps. On observe que $n(t)/t$ converge vers $d = 0.865$.



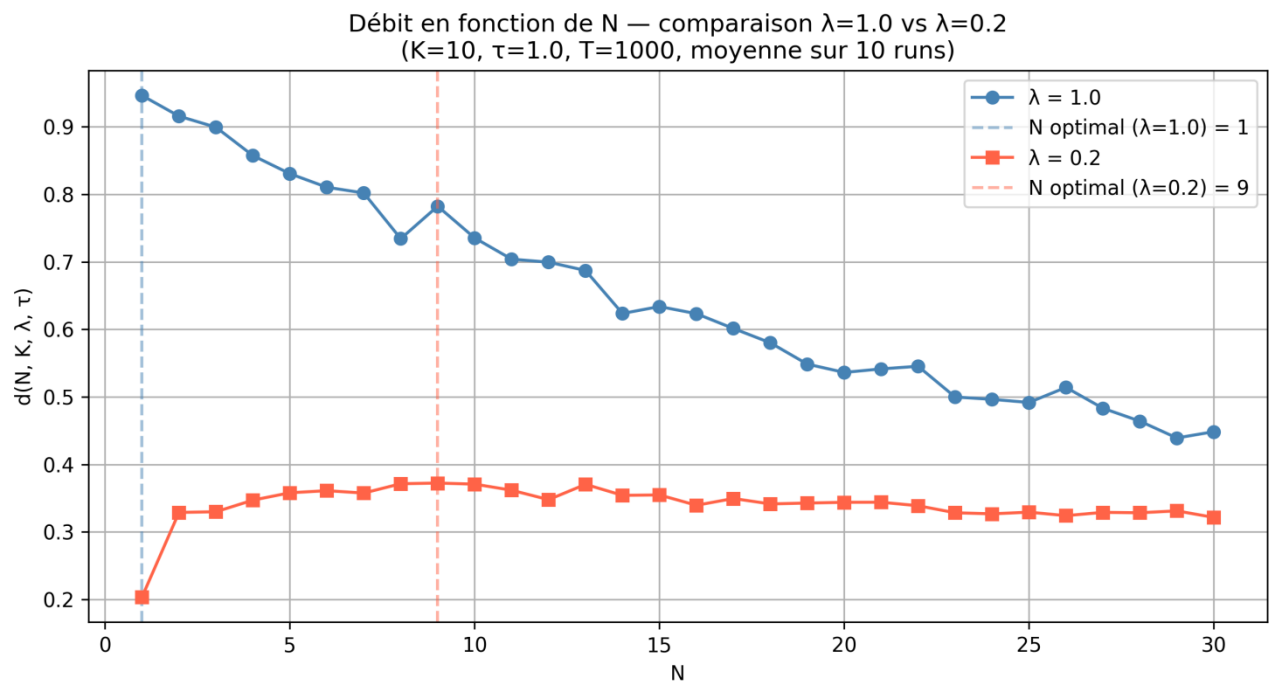
4.2 Débit en fonction de lambda (N=5, K=10, tau=1.0)

Le débit augmente avec λ jusqu'à un seuil, puis se stabilise autour de 0.9. Cette saturation s'explique par la capacité maximale du canal, fixée à 1 paquet par unité de temps : même si λ augmente encore, l'Exponential Backoff permet de maintenir un débit proche de 1. Le canal ne peut pas traiter plus d'un paquet par unité de temps, ce qui impose une borne supérieure de 1.



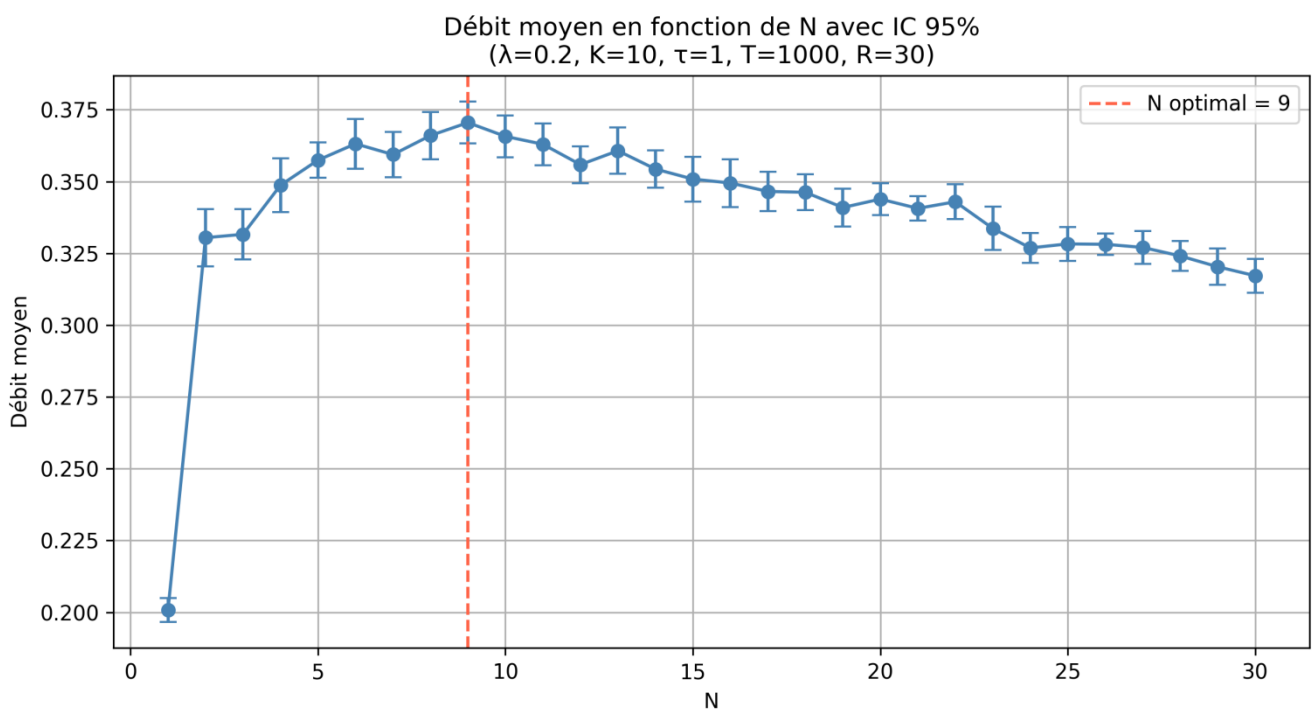
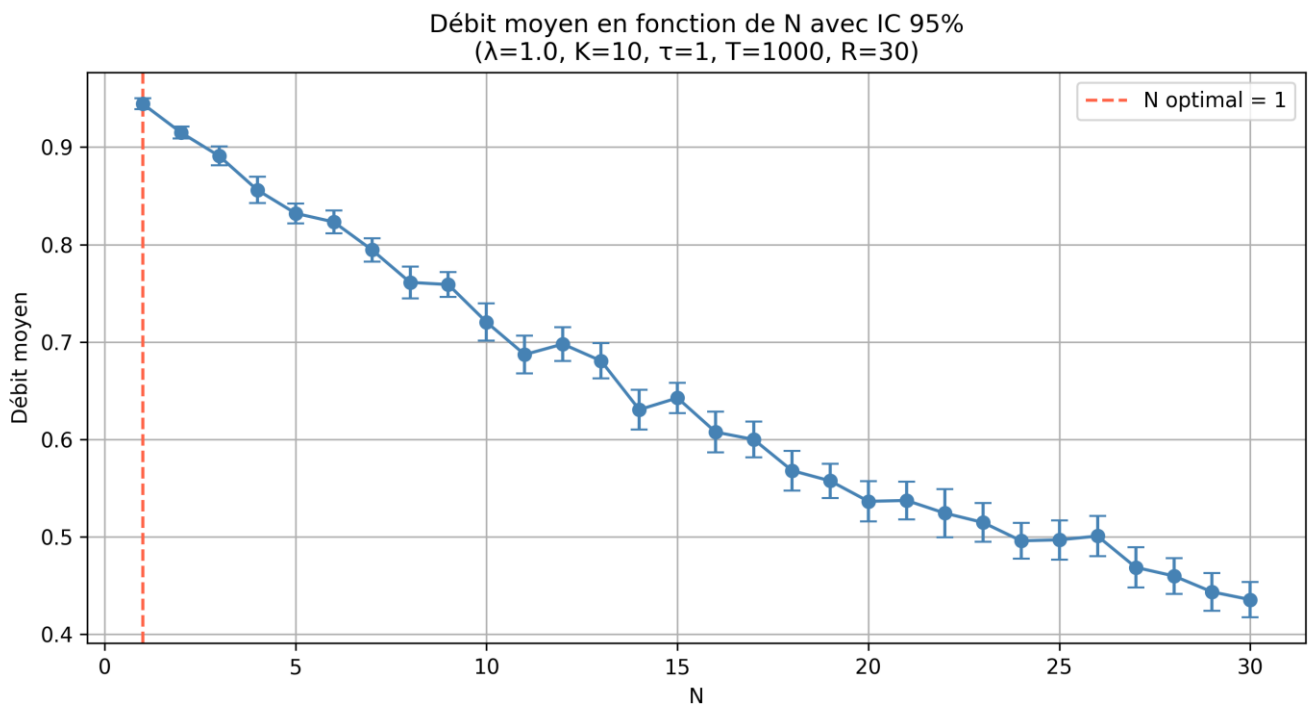
4.3 Débit en fonction de N (K=10, lambda=1.0 et 0.2, tau=1.0)

Lorsque λ est élevé ($\lambda=1.0$), le canal est déjà saturé par une seule station : le débit décroît donc de manière monotone dès $N=1$. En revanche, lorsque λ est faible ($\lambda=0.2$), augmenter le nombre de stations accroît le trafic utile disponible : le débit croît d'abord, atteint un maximum autour de $N=9$ ($d \approx 0.372$), puis diminue car les collisions deviennent trop fréquentes. Cette comparaison montre que la valeur de N qui maximise le débit dépend fortement du paramètre λ .



4.4 N optimal avec intervalle de confiance à 95%

Pour déterminer le N qui maximise le débit, on effectue $R=30$ répétitions indépendantes pour chaque valeur de N. Le N optimal estimé est $N=1$. Ce résultat s'explique par le choix de $\lambda=1.0$: une seule station génère déjà en moyenne 1 paquet par unité de temps, ce qui suffit à saturer le canal. Toute station supplémentaire ne fait qu'introduire des collisions sans apporter de trafic utile supplémentaire. Mais avec un paramètre λ plus faible ($\lambda=0.2$) conduit à un N optimal non trivial ($N=9$). Pour $\lambda=1.0$, le N optimal est $N=1$ (IC95=[0.9388, 0.9501]), car le canal est déjà saturé par une seule station. Pour $\lambda=0.2$, le N optimal est $N=9$ (IC95=[0.3632, 0.3778]), illustrant l'existence d'un vrai compromis entre trafic utile et collisions.



5. Tests de validation théorique

Pour vérifier la correction du simulateur, nous comparons les résultats à des valeurs théoriques calculables dans des cas limites.

Test 1 : $N=1$ - aucune collision possible

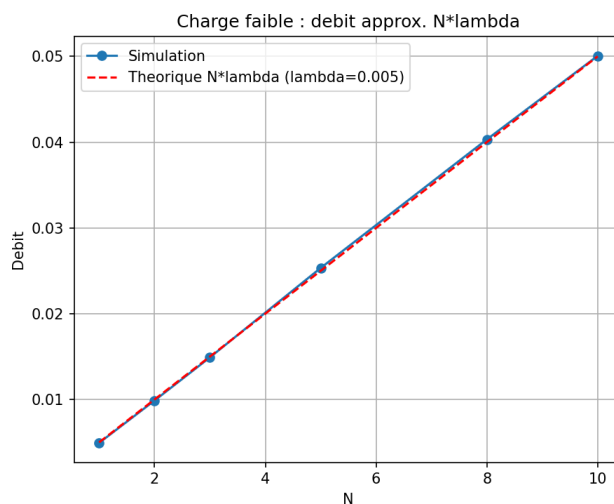
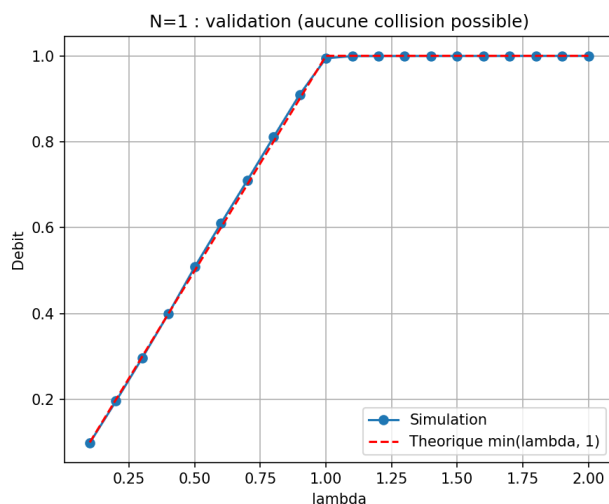
Avec une seule station, il est impossible d'avoir deux transmissions simultanées. Le débit doit être égal à $\min(\lambda, 1)$: si $\lambda < 1$, le canal n'est pas saturé ; si $\lambda \geq 1$, le canal est à pleine capacité.

Test 2 : Charge très faible ($N \cdot \lambda \ll 1$)

Quand le trafic total $N \cdot \lambda$ est très faible, la probabilité que deux stations émettent simultanément est négligeable. Le débit doit alors être proche de $N \cdot \lambda$.

Test 3 : Loi de conservation

Après le correctif du bug (les paquets en collision sont retransmis, non perdus), chaque paquet non rejeté à l'arrivée doit finir par être émis avec succès. On doit donc avoir : succès + pertes_file $\approx$ arrivées. Vérification ($T=10000$) : arrivées=49970, succès+pertes=49920, écart=50 (paquets encore en file à $t=T$).



Résultats numériques

Cas de test	Simulé	Théorique	Écart
N=1, lam=0.3	0.29660	0.30000	1.13%
N=1, lam=0.7	0.70960	0.70000	1.37%
N=1, lam=1.5	0.99970	1.00000	0.03%
N=5, lam=0.005	0.02533	0.02500	1.34%
N=10, lam=0.005	0.05006	0.05000	0.12%

Vérification collisions N=1 : 0 collision(s) sur T=10000. Attendu : 0.

6. Démarche et choix des paramètres

Choix de la durée de simulation T

Nous avons choisi $T=1000$ pour les simulations simples (variation de λ et N). Cette durée est suffisante pour que $n(t)/t$ converge vers sa valeur asymptotique, comme le montre la Figure 1. Pour l'estimation du N optimal (IC95), $T=1000$ est conservé mais on effectue $R=30$ répétitions indépendantes, ce qui assure des intervalles de confiance stables.

Choix des paramètres de référence

Les paramètres $N=5$, $K=10$, $\lambda=1.0$, $\tau=1.0$ ont été choisis pour obtenir un système chargé ($\lambda=1$ signifie 1 paquet par unité de temps par station, soit une charge totale de 5 pour un canal de capacité 1), tout en restant dans un régime où le backoff est actif.

Reproductibilité

Toutes les simulations utilisent une graine (seed) fixée pour être reproductibles. Pour les expériences avec répétitions, des graines différentes sont utilisées à chaque run ($\text{seed} = 1000 \cdot N + r$) afin de garantir l'indépendance statistique des répliques.

Comparaison Exponential Backoff vs backoff constant (bonus)

Une expérience supplémentaire compare l'Exponential Backoff avec un backoff constant (délai fixé = τ à chaque collision). L'Exponential Backoff réduit les collisions répétées et améliore le débit en charge élevée, au prix d'une latence plus importante après plusieurs collisions consécutives.

7. Conclusion

Le simulateur à événements discrets développé reproduit fidèlement le comportement du protocole MAC avec Exponential Backoff. Les tests de validation confirment la correction du simulateur dans les cas limites théoriquement calculables (erreur inférieure à 2% dans tous les cas testés).

Les expériences montrent que la valeur de N qui maximise le débit dépend fortement du paramètre λ . Pour les paramètres de référence ($\lambda=1.0$), le canal est déjà saturé par une seule station : le N optimal est $N=1$ (IC95 = [0.9388, 0.9501]) et le débit décroît de manière monotone avec N . En revanche, pour $\lambda=0.2$, un N optimal non trivial de $N=9$ est identifié (IC95 = [0.3632, 0.3778]), illustrant l'existence d'un vrai compromis entre trafic utile et collisions : augmenter le nombre de stations accroît d'abord le débit, puis les collisions deviennent trop fréquentes et le dégradent.

L'Exponential Backoff se révèle supérieur au backoff constant en charge élevée car il espace les tentatives de manière adaptative, réduisant les collisions répétées au prix d'une latence plus importante après plusieurs collisions consécutives.